

Házi feladat: IDS kijátszása evasion támadásokkal

Áttekintés

Ebben a házi feladatban evasion támadásokat kell implementálni egy hálózati forgalmon tanított behatolásdetektáló rendszer (IDS) ellen. A cél a gradiens-alapú evasion támadások képességeinek és korlátainak megértése, különösen akkor, amikor domain-specifikus szemantikai követelmények korlátozzák őket.

Célok:

- Bináris osztályozó tanítása és értékelése behatolásdetektálásra
- Korlátozott támadások implementálása Projected Gradient Descent (PGD) használatával
- Domain tudás integrálása a támadások generálásába

Az NSL-KDD adathalmaz

Az NSL-KDD a KDD Cup 1999 adathalmaz finomított verziója, amelyet behatolásdetektálási kutatásokhoz terveztek. Hálózati kapcsolat rekordokat tartalmaz 41 jellemzővel, amelyek alapvető kapcsolat attribútumokat (időtartam, protokoll típus, szolgáltatás, átvitt bájtok), tartalom-alapú indikátorokat (bejelentkezési kísérletek, kompromittált állapotok), időalapú forgalmi statisztikákat (kapcsolatszámok, hibarátákat) és host-alapú hálózati mintákat fednek le. Minden kapcsolat normál forgalomként vagy különböző támadástípusok egyikeként (pl. DoS, probe, R2L, U2R) van címkézve, beleértve a tesztkészletben szereplő zero-day támadásokat is, amelyek nem szerepelnek a tanító adatokban. Az adathalmaz kategorikus és numerikus jellemzők keverékét tartalmazza.

Az adathalmaz nyilvánosan elérhető, és a feladathoz is mellékelve van az NSL-KDD mappában.

Az `nsllkdd_features.json` fájl részletes metaadatokat tartalmaz minden jellemzőről, beleértve:

- **Jellemző neve és leírása:** Magyarázat arról, hogy a jellemző mit reprezentál
- **Lehetséges értékek:** Az érvényes tartomány vagy a kategorikus értékek halmaza
- **Módosíthatósági szint:** Jelzi, hogy mely jellemzőket tudja egy támadó reálisan módosítani egy evasion támadás során anélkül, hogy érvénytelenítené magát a támadást

Fontos: Nem minden jellemző módosítható szabadon egy támadó által. Például a sikeres kompromittálást jelző jellemzők (pl. `num_root`, `num_compromised`) nem módosíthatók a támadó által kijátszás közben, mivel ezek módosítása azt jelentené, hogy a támadás már sikerült vagy elbukott. A JSON fájlban található módosíthatósági

annotációk iránymutatást adnak arra, hogy mely jellemzőket kell perturbálni a támadások során.

Feladatok

0. feladat: Adatok előfeldolgozása

Dolgozza fel az NSL-KDD tanító és teszt adathalmazokat. Az előfeldolgozási pipeline-nak a következőket kell tartalmaznia:

- **Feature információk egyesítése:** A tanító és teszt adatok feature halmazainak kombinálása a konzisztens kódolás biztosítása érdekében
- **One-hot kódolás:** A következő kategorikus jellemzők átalakítása one-hot reprezentációra: `protocol_type`, `service`, `flag`
- **Standardizálás:** `StandardScaler` alkalmazása az összes jellemző normalizálására (tanító adatokon illetve, tanító és teszt adatokon transzformálva)
- **Bináris címkék:** A multi-class támadás címkék átalakítása bináris osztályozásra (0 = normál, 1 = támadás)

Fontos: Ne végezzen semmilyen további előfeldolgozást a fent felsoroltakon túl.

1. feladat: Az IDS bináris osztályozó tanítása és értékelése

Implementálja és tanítsa a tanítóadaton a mellékelt bináris neurális hálózat architektúrát PyTorch-ban behatolásdetektálásra. A modell architektúra az `IDS.py` fájlban található.

Modell architektúra:

- Input réteg: 122 feature (one-hot kódolás után)
- - 1. rejtett réteg: 64 neuron, ReLU aktiváció, Dropout(0.3)
 - 2. rejtett réteg: 32 neuron, ReLU aktiváció, Dropout(0.3)
- Output réteg: 1 neuron, Sigmoid aktiváció

Tanítási hiperparaméterek:

- Loss függvény: Binary Cross-Entropy (BCELoss)
- Optimalizáló: Adam
- Tanulási ráta: 0.001
- Batch méret: 128
- Epoch-ok: 10

A tanítás után értékelje a modell teljesítményét a következőkön:

- **Teljes teszt halmaz:** Adja meg az általános pontosságot, precision-t, recall-t és F1-score-t!
- **Zero-day támadások:** A teszt halmazban szereplő támadások, amelyek *nem* voltak jelen a tanító adatokban. Adja meg a pontosságot ezen a részhalmazon!
 - *Mik azok a zero-day támadások?* Ezek olyan új támadási mintákat reprezentálnak, amelyekkel az IDS soha nem találkozott a tanítás során. Valós környezetben a zero-day támadások különösen veszélyesek, mert a hagyományos szignatúra-alapú detektáló rendszerek nem tudják felismerni őket. A zero-day támadásokon való értékelés az IDS azon képességét teszteli, hogy általánosítsa az ismert fenyegetéseken túl.
- **Nem zero-day támadások:** Olyan támadások, amelyek *szerepeltek* a tanításban. Adja meg a pontosságot ezen a részhalmazon!

Emlékeztető: A teszt részhalmazokon történő értékeléskor győződjön meg arról, hogy ugyanazokat az előfeldolgozási lépéseket hajtja végre mint a tanítás során, valamint pontosan ugyanazt a standardizálást alkalmazza (a tanító adatokon illesztett scaler használatával)!

2. feladat: Evasion korlátozott PGD-vel

Implementálja a Projected Gradient Descent (PGD) támadást a tanított IDS kijátszására, a következő fontos korlátokkal és követelményekkel:

Támadás beállítása:

- Csak olyan teszt mintákat próbáljon megtámadni, amelyeket az IDS **helyesen támadásként osztályoz**. (Nincs értelme olyan mintákat támadni, amelyeket az IDS már eleve rosszul osztályoz.)
- Cél: módosítsa ezeket a mintákat úgy, hogy megtévevessze az IDS-t, és normál forgalomként osztályozza őket!

Módosíthatósági korlátok:

- Csak azokat a jellemzőket módosítsa, amelyek erősen (highly) módosíthatóként vannak megjelölve az `ns_lkdd_features.json` fájlban. A sikeres kompromittálást jelző jellemzők nem módosíthatók anélkül, hogy érvénytelenítené a támadást. Amennyiben nincs sikeres támadás, használjon részlegesen (partially) módosítható jellemzőket!
- Minden PGD iteráció után vágja le a perturbált jellemzőket az érvényes tartományukra, amelyet a tanító adatokból számított. Konkrétan: számítsa ki minden jellemző min/max értékét a tanító adatokból, és ezekkel clamp-elje a perturbált mintákat.

Tesztelendő epsilon értékek: 0.05, 0.1, 0.15, 0.2, 0.3

PGD hiperparaméterek:

- Iterációk száma: 40
- Lépésköz (alpha): 0.01
- Inicializálás: Kezdje az eredeti mintától (nincs random inicializálás ennél a feladatnál)

Minden epsilon értékre adja meg:

- **Sikeres támadások száma:** Hány helyesen osztályozott támadást sikerült megtéveszteni, hogy normálként osztályozzák?
- **Plauzibilis támadások aránya:** A sikeres kijátszások közül mekkora hányad megy át a plauzibilitási ellenőrzéseken?

Plauzibilitás ellenőrzése:

Használja a mellékelt `simple_rules.py` szkriptet annak ellenőrzésére, hogy az adversariális példák nem sértik-e a kritikus domain szabályokat. Például:

- Hosszú időtartam nulla átvitt bájjal gyanús
- Nagyon rövid időtartam hatalmas adatátvitellel irreális
- Magas hibatéráshoz elegendő kapcsolatszám szükséges

Egy *plauzibilis támadás* olyan, amely sikeresen kijátssza az IDS-t és átmegy minden plauzibilitási szabályon, vagyis egy reális, működő támadást reprezentál, amely átcsúszik a detektáláson.

Miért fontos ez: A gyakorlatban egy olyan támadási példa, amely ugyan becsapja az osztályozót, de sérti a hálózati forgalom fizikáját (pl. 1GB átvitele 0.001 másodperc alatt), használhatatlan és nem fog ténylegesen működni a gyakorlatban. Ez a feladat rávilágít a matematikailag ugyan helyes támadói minták és a valós támadások közötti különbségre.

3. feladat: PGD plauzibilitási loss-szal

Javítsa a PGD támadásodat egy plauzibilitási tag beépítésével a loss függvénybe. Ez a megközelítés közvetlenül integrálja a domain tudást az optimalizációba, ahelyett hogy csak utólag szűrné az eredményeket.

1. lépés: Gaussian Naive Bayes (GNB) modell tanítása

- Tanítsa a GNB-t a **tanító adatokon** az **eredeti (multi-class) támadás címkék** használatával, nem bináris címkékkel
- A GNB minden feature-t független Gauss-eloszlásként modellez osztályonként, így egy egyszerű valószínűségi modellt kapunk arról, hogy milyen a "normál" támadási forgalom minden támadástípusra

- **Korlát:** Ez a modell csak a tanítás során látott támadástípusokra tud likelihood-okat szolgáltatni. Ezért ez a továbbfejlesztett PGD nem alkalmazható a teszt halmazban szereplő zero-day támadásokra.

2. lépés: A kombinált loss definiálása

Ahelyett, hogy csak a normálként való osztályozás valószínűségét maximalizálnánk, most a következőt optimalizáljuk:

$$L_{\text{total}} = L_{\text{classifier}} + \lambda \times L_{\text{plausibility}}$$

ahol:

- $L_{\text{classifier}}$ a bináris cross-entropy loss a cél (normál) osztályra vonatkozóan. Ennek minimalizálása megpróbálja becsapni az IDS-t.
- $L_{\text{plausibility}}$ az támadói minta *negatív* log-likelihood-ja a GNB modell alatt, az **eredeti támadás címkére** kondicionálva. Ennek minimalizálása a mintát közel tartja az érvényes támadási manifoldokhoz.
- λ (lambda) egyensúlyozza a két célt. Használjon $\lambda = 0.1$ -et ehhez a feladathoz!

Intuíció: A classifier loss eltávolítja a példát a támadási manifoldtól (a normál osztály felé). A plauzibilitási loss visszahúzza a támadási manifold felé (hogy reális maradjon). A kombinált loss egy optimális pontot talál: elég közel a döntési határhoz, hogy becsapja az IDS-t, de még mindig hasonlítson egy érvényes támadásra.

3. lépés: A plauzibilitási loss kiszámítása

Gaussian Naive Bayes esetén egy minta y osztályra vonatkozó log-likelihood-ja:

$$\log p(y) = \sum_i \log p(x_i|y)$$

ahol minden i jellemző a következőképpen van modellezve:

$$p(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_i^2}} \exp\left(-\frac{(x_i - \mu_i)^2}{2\sigma_i^2}\right)$$

ahol μ_i és σ_i az i jellemző átlaga és szórása az y osztályra (a tanított GNB modellből kinyerve).

A $\log p(x_i|y)$ gradiense az x -et a μ osztályátlag felé húzza, arra ösztönözve az támadói mintát, hogy az eredeti támadástípus tipikus jellemző eloszlásán belül maradjon.

4. lépés: Implementáció és értékelés

- Implementálja a PGD-t a kombinált loss függvényvel!
- Futtassa ugyanazon a helyesen osztályozott teszt támadások halmazán, mint a 2. feladatban (a zero-day támadások kizárásával)!
- Használja ugyanazokat az epsilon értékeket, mint a 2. feladatban!
- Minden epsilon-ra adja meg:
 - A sikeres kijátszások számát
 - A plauzibilis támadások arányát (ugyanazt a plauzibilitás ellenőrzőt használva, mint a 2. feladatban)

4. feladat: Elemzési kérdések

Válaszoljon a következő kérdésekre a beadandódban:

1. Hasonlítsa össze a 2. és 3. feladat eredményeit: javítja-e a plauzibilitás-tudatos loss a plauzibilis támadások arányát? Miért?
2. Miért jobb a plauzibilitás-tudatos loss-szal működő PGD (3. feladat) a következő alternatíváknál?
 - *A alternatíva - Utólagos szűrés:* A 2. feladatból származó korlátozott PGD futtatása, majd a generált támadói minták közül azok elvetése utólag, amelyek megsértik a plauzibilitási ellenőrzéseket.
 - *B alternatíva - Lépésenkénti elutasítás:* Plauzibilitási ellenőrzések végrehajtása minden gradiens descent lépés után a támadás során, és azon lépések elutasítása, amelyek implauzibilis példákhoz vezetnének.

Beadási útmutató

Adjon be egy Jupyter notebookot vagy Python szkriptet, amely tartalmazza:

- Az összes feladat teljes implementációját
- Világos dokumentációt és kommenteket a kódod magyarázatához
- Eredménytáblázatokat, amelyek mutatják:
 - Az IDS pontosságát a teszt halmazon, zero-day támadásokon és nem zero-day támadásokon
 - A 2. és 3. feladatra: sikeres kijátszások számát és plauzibilis támadások arányát minden epsilon értékre
- Rövid elemzést (4. feladat)
- Csapatonként elég egy beadás